

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

ЗВІТ

з лабораторної роботи №1

з дисципліни «Архітектура програмного забезпечення»

на тему: «РОЗРОБКА VISION & SCOPE»

Виконала:

ст. гр. ПЗП-22-9

М'яч Катерина Олександрівна

Перевірив:

Дашенков Дмитро Сергійович

Харків 2025

Vision and Scope Document

for

Програмна система для моніторингу запасів та зберігання меду

Version 1.0 approved

Prepared by М'яч Катерина

ХНУРЕ

01.06.2025

Table of Contents

Table of Contents 3

Revision History 3

1. Business Requirements 4

 1.1. Background 4

 1.2. Business Opportunity 4

2. Vision of the Solution 5

 2.1. Vision Statement 5

 2.2. Major Features 6

3. Scope and Limitations 6

 3.1. Scope of Initial Release 6

 3.2. Scope of Subsequent Releases 8

4. Business Context 9

 4.3. Operating Environment 9

Revision History

Name	Date	Reason For Changes	Version
Початковий випуск	01.06.2025		Розробка початкового документа 1.0

1. Business Requirements

1.1. Background

Бджільництво є важливою галуззю сільського господарства, а мед – цінним продуктом. Проте пасічники, особливо малі та середні господарства, часто стикаються з низкою проблем при управлінні запасами меду та контролі умов його зберігання:

- 1. Неефективний облік: ручний облік запасів меду (кількість, сорти, дати збору, місця зберігання) є трудомістким, схильним до помилок та ускладнює швидке отримання актуальної інформації.*
- 2. Ризик псування продукції: мед вимагає певних умов зберігання (температура, вологість). Відсутність належного моніторингу цих параметрів може призвести до втрати якості, кристалізації або навіть псування продукції.*
- 3. Складність планування: без точних даних про наявні запаси та їх стан, пасічникам важко планувати реалізацію продукції, закупівлю тари та інші операційні аспекти.*
- 4. Відсутність інтеграції даних з IoT-пристроїв: наявність IoT-пристроїв для моніторингу умов зберігання є кроком вперед, але без централізованої системи для збору, аналізу та візуалізації цих даних, їх потенціал використовується не повністю. Користувачам потрібен зручний інтерфейс для доступу до цієї інформації.*

Пропонована програмна система покликана вирішити ці проблеми, надаючи пасічникам інструмент для автоматизованого моніторингу запасів меду та умов його зберігання, інтегруючи дані з IoT-пристроїв та надаючи зручний доступ через веб- та мобільний додатки.

1.2. Business Opportunity

На ринку існують різноманітні рішення, які частково покривають потреби пасічників:

- 1. Загальні системи управління складом (Inventory Management Systems):*
 - Приклади: Odoo Inventory, Zoho Inventory, QuickBooks Commerce.*
 - Переваги: розвинений функціонал для обліку товарів, управління замовленнями.*
 - Недоліки: часто є надлишковими та складними для малих пасік, не мають специфічного функціоналу для меду (наприклад, моніторинг умов зберігання з IoT, облік сортів,*

дат збору в контексті бджільництва). Вимагають значної кастомізації.

2. Спеціалізовані IoT-рішення для моніторингу вуликів/сховищ:
 - Приклади: *BroodMinder, SolutionBee, Arnia*.
 - Переваги: спеціалізуються на зборі даних з датчиків (температура, вологість, вага вулика).
 - Недоліки: часто фокусуються лише на зборі даних, а не на комплексному управлінні запасами готової продукції (меду). Веб- та мобільні інтерфейси можуть бути обмеженими або відсутніми для управління саме запасами.
3. Прості табличні редактори та кастомні рішення:
 - Приклади: *Microsoft Excel, Google Sheets*.
 - Переваги: доступність, гнучкість.
 - Недоліки: відсутність автоматизації, складність інтеграції з IoT, високий ризик помилок, відсутність сповіщень та зручного мобільного доступу.

Ця система прагне поєднати переваги: забезпечити простий, але функціональний облік запасів меду, інтегрувати дані з існуючих або розроблюваних IoT-пристроїв для моніторингу умов зберігання, та надати зручний доступ через веб- та мобільні платформи.

2. Vision of the Solution

2.1. Vision Statement

Програмна система "HoneyTrack" призначена для пасічників та підприємств, що займаються виробництвом та зберіганням меду.

Головні цілі системи:

- Забезпечити точний та актуальний облік запасів меду (кількість, сорт, дата збору, місце зберігання).
- Інтегрувати дані з IoT-пристроїв для моніторингу умов зберігання меду (температура, вологість) в реальному часі.
- Надавати користувачам зручний доступ до інформації через веб-інтерфейс та мобільний додаток.
- Сповіщати користувачів про критичні зміни умов зберігання.
- Допомогти оптимізувати процеси зберігання та реалізації меду, зменшити втрати продукції.

2.2. Major Features

Система функціонуватиме наступним чином:

1. *Збір даних з IoT-пристроїв: IoT-пристрої, розміщені в місцях зберігання меду (або безпосередньо в тарі з медом), періодично вимірюють параметри мікроклімату (наприклад, температура, вологість) та передають ці дані на сервер.*
2. *Введення даних про запаси: користувач через веб-інтерфейс або мобільний додаток вносить інформацію про партії меду:*
 - *Назва/ідентифікатор партії.*
 - *Сорт меду.*
 - *Дата збору/фасування.*
 - *Кількість (наприклад, кг, кількість банок).*
 - *Місце зберігання (з можливістю прив'язки до конкретного IoT-датчика).*
 - *Додаткові нотатки.*
3. *Обробка та зберігання даних: серверна частина отримує дані від IoT-пристроїв та користувачів, обробляє їх та зберігає в базі даних (SQLite).*
4. *Візуалізація та доступ до даних:*
 - *Веб-клієнт: надає детальну інформаційну панель (dashboard) з оглядом запасів, поточними та історичними даними з датчиків (графіки), можливість додавати, редагувати та видаляти записи про мед.*
 - *Мобільний клієнт: надає швидкий доступ до ключової інформації про запаси та умови зберігання, можливість отримувати push-сповіщення.*
5. *Система сповіщень: якщо параметри зберігання (отримані з IoT-датчиків) виходять за встановлені користувачем допустимі межі, система генерує сповіщення, які відображаються в веб-інтерфейсі та надсилаються як push-сповіщення на мобільний додаток.*
6. *Управління користувачами: базова система реєстрації та авторизації для доступу до даних.*

3. Scope and Limitations

3.1. Scope of Initial Release

IoT Клієнт:

- *Функції:*
 - *Збір даних з датчиків (температура, вологість).*

- *Періодична передача зібраних даних на серверний API (наприклад, через HTTP POST запити у форматі JSON).*
- *Можливість ідентифікації датчика (унікальний ID).*

- *Вимоги:*

- *Стабільне підключення до мережі (Wi-Fi).*
- *Низьке енергоспоживання (якщо працює від батарей).*
- *Надійність передачі даних.*

Серверна частина (Backend):

- *Функції:*

- *API для IoT-клієнтів: прийом даних з датчиків, їх валідація та збереження.*
- *API для веб- та мобільних клієнтів (RESTful API):*
 - *Аутентифікація та авторизація користувачів (наприклад, через JWT).*
 - *CRUD-операції (Create, Read, Update, Delete) для запасів меду.*
 - *Отримання списку запасів з можливістю фільтрації та сортування.*
 - *Отримання поточних та історичних даних з датчиків для конкретного місця зберігання/партії.*
 - *Налаштування порогових значень для сповіщень.*
- *Зберігання даних: взаємодія з базою даних SQLite для збереження інформації про користувачів, запаси меду, дані з датчиків, налаштування.*
- *Логіка сповіщень: аналіз вхідних даних з датчиків, порівняння їх з пороговими значеннями та генерація сповіщень.*

- *Вимоги:*

- *Надійність та стабільність роботи.*
- *Масштабованість.*
- *Безпека даних та API.*

3.2. Scope of Subsequent Releases

Веб-клієнт (Frontend - React/JS):

- *Функції:*
 - *Реєстрація та авторизація користувача.*
 - *Інформаційна панель (Dashboard):*
 - *Загальний огляд кількості запасів меду.*
 - *Відображення поточних показань датчиків для ключових місць зберігання.*
 - *Список останніх сповіщень.*
 - *Управління запасами:*
 - *Перегляд списку всіх партій меду з детальною інформацією (сорт, дата, кількість, місце зберігання, поточні умови).*
 - *Додавання нової партії меду.*
 - *Редагування інформації про існуючу партію.*
 - *Видалення партії.*
 - *Фільтрація та пошук по запасах.*
 - *Моніторинг умов зберігання:*
 - *Відображення історичних даних з датчиків у вигляді графіків (температура, вологість за період).*
 - *Можливість вибору конкретного датчика/місця зберігання для перегляду.*
 - *Налаштування сповіщень:*
 - *Встановлення порогових значень температури та вологості для кожного місця зберігання/типу меду.*
 - *Перегляд списку активних та архівних сповіщень.*
- *Вимоги:*
 - *Інтуїтивно зрозумілий та зручний інтерфейс користувача (UI/UX).*
 - *Адаптивний дизайн для різних розмірів екранів (десктоп, планшет).*
 - *Швидке завантаження та реакція на дії користувача.*

Мобільний клієнт (Kotlin):

- *Функції:*
 - *Реєстрація та авторизація користувача (можливо, через той самий бекенд, що й веб).*
 - *Головний екран:*

- Швидкий огляд ключових показників (загальна кількість меду, кількість критичних сповіщень).
 - Список останніх сповіщень.
- Перегляд запасів:
 - Спрощений список партій меду з основною інформацією.
 - Можливість перегляду деталей партії та поточних умов зберігання.
- Сповіщення:
 - Отримання push-сповіщень про вихід параметрів за допустимі межі.
 - Перегляд списку сповіщень.
- Вимоги:
 - Оптимізований для мобільних пристроїв інтерфейс.
 - Ефективне використання ресурсів мобільного пристрою.
 - Підтримка push-сповіщень.
 - Робота в офлайн-режимі з обмеженим функціоналом (наприклад, кешування даних для перегляду).

4. Business Context

4.1. Operating Environment

IoT Клієнт:

- Апаратна платформа: (наприклад, ESP32)
- Датчики: (наприклад, DHT22/DHT11 для температури та вологості)
- Програмне забезпечення: C/C++ (Arduino IDE, PlatformIO), MicroPython.

Серверна частина (Backend):

- Мова програмування: JavaScript (Node.js).
- Фреймворк: Express.js.
- База даних: SQLite.
- API тестування: Postman або Insomnia.

Веб-клієнт (Frontend):

- Мова програмування: JavaScript.

- *Бібліотека/Фреймворк: React.js.*
- *Верстка: HTML5, CSS3.*

Мобільний клієнт:

- *Мова програмування: Kotlin.*
- *Інструменти: Android Studio.*